Assignment #1

Embodied Artificial Intelligence (CS1050)

Name: Jiang Jiahao **Student ID:** 22671064

Instructor: Professor Gaoang Wang **Term:** Fall 2026

Due: October 30, 2026, 23:59 (China Standard Time)

Question 1

The two lines have the common direction $\mathbf{d} = (4, 2, 5)^\top$. For a point $\mathbf{a} + \lambda\mathbf{d}$ on either line,

$$\lim_{\lambda\to\infty} \left(\frac{a_X + 4\lambda}{a_Z + 5\lambda}, \frac{a_Y + 2\lambda}{a_Z + 5\lambda} \right) = \left(\frac{4}{5}, \frac{2}{5} \right).$$

Thus both lines have the same vanishing point $\boxed{\mathbf{v} = (4/5,\ 2/5)^\top = (0.8,\ 0.4)^\top}$. Its homogeneous image coordinates are $(4, 2, 5)^\top$; the different line origins do not affect this limit.

Question 2

The plane has normal $\mathbf{n} = (6, 3, 2)^\top$. Every direction $\mathbf{d}$ parallel to it satisfies $\mathbf{n}^\top\mathbf{d} = 0$. Under normalized perspective projection, its vanishing point is $\mathbf{v} \sim (d_X, d_Y, d_Z)^\top$. Therefore all these points satisfy

$$\mathbf{n}^\top\mathbf{v} = 0 \quad \Longrightarrow \quad \boxed{6x + 3y + 2 = 0}, \qquad \boxed{\ell_\infty \sim (6, 3, 2)^\top}.$$

The offset 17 determines the plane's position, whereas its vanishing line depends only on its orientation. Directions with $d_Z = 0$ are included as homogeneous points at infinity.

Question 3

For $12X + 8Y + 28Z = 23$, a normal is

$$\mathbf{n} = (12, 8, 28)^\top = 4(3, 2, 7)^\top.$$

If $\mathbf{u}$ and $\mathbf{v}$ lie on the plane, subtracting their plane equations gives $\mathbf{n}^\top(\mathbf{u} - \mathbf{v}) = 0$. Consequently $(3, 2, 7)^\top$ is orthogonal to every direction in the plane, so it is perpendicular to the plane.

Question 4

The shared translation $\mathbf{t} = (-3, 2, 1)^\top$ cancels: $\mathbf{x}_2 - \mathbf{y}_2 = R(\mathbf{x}_1 - \mathbf{y}_1)$. Writing $\mathbf{d} = \mathbf{x}_1 - \mathbf{y}_1$ and using $R^\top R = I$,

$$\|\mathbf{x}_2 - \mathbf{y}_2\|_2^2 = \mathbf{d}^\top R^\top R\mathbf{d} = \mathbf{d}^\top\mathbf{d} = \|\mathbf{x}_1 - \mathbf{y}_1\|_2^2.$$

Both norms are nonnegative; taking square roots proves $\boxed{\|\mathbf{x}_2 - \mathbf{y}_2\|_2 = \|\mathbf{x}_1 - \mathbf{y}_1\|_2}$.

Question 5

Let $\mathbf{X}_i = (X_i, Y_i, Z_i, 1)^\top$ and denote the rows of P by $\mathbf{p}_1^\top, \mathbf{p}_2^\top, \mathbf{p}_3^\top$. Each correspondence imposes the two linear constraints

$$(\mathbf{p}_1^\top - x_i \mathbf{p}_3^\top)\mathbf{X}_i = 0, \qquad (\mathbf{p}_2^\top - y_i \mathbf{p}_3^\top)\mathbf{X}_i = 0.$$

Stacking the six pairs produces $A\mathbf{p} = 0$, where A is 12×12 and $\mathbf{p}$ contains the rows of P in order. Its rank is 11, so the solution is unique up to a nonzero scale. The exact solution has representative

$$P_0 = \begin{bmatrix} 180 & 0 & 0 & 280 \\ 0 & 180 & 0 & 220 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \|P_0\|_F = \sqrt{180^2 + 280^2 + 180^2 + 220^2 + 1} = \sqrt{191601}.$$

The requested normalization is

$$\boxed{P = \frac{1}{\sqrt{191601}} \begin{bmatrix} 180 & 0 & 0 & 280 \\ 0 & 180 & 0 & 220 \\ 0 & 0 & 0 & 1 \end{bmatrix}}.$$

This camera gives $x = 180X + 280$, $y = 180Y + 220$, independent of Z: it is an affine camera. In particular, the fifth point gives $(370, 310)$ and the sixth gives $(370, 220)$. A finite pinhole intrinsic matrix is not an additional assumption of this computation. The matrices P and $-P$ have the same Frobenius norm and project to the same image points; the implementation fixes the sign by making the largest-magnitude entry positive.

Implementation. `experients/assignment1/q5_camera.py`, reproduced in Appendix A. The code centers and scales the 3D and 2D points, takes the last right singular vector of the normalized design matrix, and denormalizes with $P = T_{2D}^{-1} P_n T_{3D}$ before applying the Frobenius normalization. The following is captured from the script's actual `print` output.

```
Q5: normalized DLT from the six given correspondences
P (Frobenius norm = 1; largest coefficient positive):
[[ 0.411219423  -0.           -0.           0.639674658 ]
 [ 0.            0.411219423  -0.           0.502601517 ]
 [ 0.           -0.           -0.           0.0022845524]]
Frobenius norm: 1.000000000000
Normalized design matrix rank: 11 / 12
Normalized design singular values:
[5.6880816074 4.0107738785 3.9540185873 3.4076114648 3.0157576053 1.8739225228
 1.6608812177 1.6302286554 1.5086727349 1.4111622674 1.4049726386 0.          ]
Reprojected pixels (columns match the assignment):
[[280. 460. 280. 460. 370. 370.]
 [220. 220. 400. 400. 310. 220.]]
Maximum reprojection error (px): 1.798e-13
P / P[2, 3] (a readable representative):
[[180.  -0.  -0. 280.]
 [  0. 180.  -0. 220.]
 [  0.  -0.  -0.   1.]]
```

Question 6

Let $\mathbf{X}_h = (X, Y, Z, W)^\top$. For each camera, stack the rows $x_j P_3^{(j)} - P_1^{(j)}$ and $y_j P_3^{(j)} - P_2^{(j)}$. The supplied matrices and pixels give

$$A = \begin{bmatrix} -900 & 0 & 0 & 0 \\ 0 & -900 & 0 & 0 \\ -900 & 0 & -15 & 6 \\ 0 & -900 & 0 & 0 \end{bmatrix}, \qquad A\mathbf{X}_h = 0.$$

The first two equations imply $X = Y = 0$; the third then gives $-15Z + 6W = 0$. Choosing $W = 1$ yields

$$\boxed{\mathbf{X} = (0,\ 0,\ 0.4)^\top}, \qquad \mathbf{X}_h = (0,\ 0,\ 0.4,\ 1)^\top.$$

The fourth column of $P^{(2)}$ is already expressed in projection-matrix units. Indeed, $P^{(2)} = K[I \mid \mathbf{t}]$ has $\mathbf{t} = (-6/900, 0, 0)^\top$ and camera center $\mathbf{C}_2 = (6/900, 0, 0)^\top$. Hence the physical baseline is $B = 6/900 = 1/150$ in the given world units, and the disparity check is

$$Z = \frac{fB}{x_1 - x_2} = \frac{900(6/900)}{400 - 385} = 0.4.$$

Both camera depths are positive. Direct reprojection confirms

$$\pi(P^{(1)}\mathbf{X}_h) = (400, 250), \qquad \pi(P^{(2)}\mathbf{X}_h) = (385, 250), \quad \pi(a, b, c) = (a/c, b/c).$$

Implementation. `experients/assignment1/q6_triangulation.py`, reproduced in Appendix B, solves the homogeneous system by SVD and divides by the last coordinate. Its captured `print` output is:

```
Q6: homogeneous linear triangulation
3D point [X, Y, Z]:
[0.  0.  0.4]
Homogeneous point [X, Y, Z, 1]:
[0.  0.  0.4 1. ]
Design singular values:
[1272.8434744746 1272.7922061358   11.4231995299    0.          ]
Reprojected pixels (rows: view 1, view 2):
[[400. 250.]
 [385. 250.]]
Maximum reprojection error (px): 2.842e-13
Depth in both cameras: 0.4000000000
Independent disparity depth 6 / (400 - 385): 0.4000000000
```

Question 7

With the convention $\mathbf{X}_2 = R\mathbf{X}_1 + \mathbf{t}$, corresponding calibrated image points obey $\mathbf{p}_2^\top E \mathbf{p}_1 = 0$, with

$$E = [\mathbf{t}]_\times R, \qquad [\mathbf{t}]_\times = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}.$$

For $R = I$ and $\mathbf{t} = (3, 0, 0)^\top$,

$$\boxed{E = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -3 \\ 0 & 3 & 0 \end{bmatrix}}.$$

It is defined up to nonzero scale. Since $K_1 = K_2 = I$, the fundamental matrix $F = K_2^{-\top} E K_1^{-1}$ is also E.

Question 8

The epipolar line in image 2 is $\boldsymbol{\ell}_2 = E\mathbf{p}_1$:

$$\boldsymbol{\ell}_2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -3 \\ 0 & 3 & 0 \end{bmatrix} \begin{bmatrix} 280 \\ 200 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -3 \\ 600 \end{bmatrix}.$$

Thus $\boxed{\boldsymbol{\ell}_2 \sim (0, 1, -200)^\top}$, or equivalently $\boxed{y_2 = 200}$, since $0x_2 - 3y_2 + 600 = 0$.

Question 9

First multiply and then divide by the homogeneous image coordinate:

$$P\mathbf{X} = \begin{bmatrix} 3 \\ 1 \\ 2 \end{bmatrix} \quad \Longrightarrow \quad \widehat{\mathbf{x}} = (1.5,\, 0.5)^\top.$$

Using the supplied projection matrix and measurement directly, in the pixel units requested by the question, the Euclidean reprojection error is

$$\begin{aligned} e &= \|\widehat{\mathbf{x}} - \mathbf{x}_m\|_2 = \sqrt{(1.5 - 280)^2 + (0.5 - 200)^2} \\ &= \sqrt{278.5^2 + 199.5^2} = \sqrt{117362.5} \approx \boxed{342.5821 \text{ px}}. \end{aligned}$$

No extra intrinsic matrix or coordinate conversion is specified.

Question 10

Unit-scale orthographic projection along $+Z$ discards the depth coordinate:

$$\mathbf{x} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 5 \\ 4 \\ 8 \end{bmatrix} = \boxed{\begin{bmatrix} 5 \\ 4 \end{bmatrix}}.$$

Appendix A: complete Python code for Question 5

File: `experients/assignment1/q5_camera.py`. This is the same source file that generated the output in Question 5.

```python
"""Question 5: normalized 3D-to-2D DLT, using the given six columns."""

import numpy as np


WORLD = np.array([
    [0, 1, 0, 1, 0.5, 0.5],
    [0, 0, 1, 1, 0.5, 0],
    [2, 2, 2, 2, 4, 4],
], dtype=float)
IMAGE = np.array([
    [280, 460, 280, 460, 370, 370],
    [220, 220, 400, 400, 310, 220],
], dtype=float)


def normalize_points(points):
    """Center points and set mean distance to sqrt(dimension)."""
    dimension = points.shape[0]
    center = points.mean(axis=1)
    distance = np.linalg.norm(points - center[:, None], axis=0).mean()
    if distance <= np.finfo(float).eps:
        raise ValueError("Coincident points cannot be normalized.")
    scale = np.sqrt(dimension) / distance
    transform = np.eye(dimension + 1)
    transform[:dimension, :dimension] *= scale
    transform[:dimension, -1] = -scale * center
    homogeneous = np.vstack((points, np.ones(points.shape[1])))
    return transform @ homogeneous, transform


def estimate_camera(world=WORLD, image=IMAGE):
    """Solve A p = 0, denormalize, fix sign, and set ||P||_F = 1."""
    if world.shape != (3, 6) or image.shape != (2, 6):
        raise ValueError("This exercise expects six 3D/2D correspondences.")
    world_n, world_transform = normalize_points(world)
    image_n, image_transform = normalize_points(image)
    rows = []
    for point, pixel in zip(world_n.T, image_n.T):
        u, v = pixel[:2] / pixel[2]
        rows.append(np.r_[point, np.zeros(4), -u * point])
        rows.append(np.r_[np.zeros(4), point, -v * point])
    design = np.array(rows)
    _, singular_values, vh = np.linalg.svd(design)
    rank = np.linalg.matrix_rank(design)
    if rank != 11:
        raise ValueError(f"Expected a unique camera up to scale; rank={rank}.")
    camera_n = vh[-1].reshape(3, 4)
    camera = np.linalg.solve(image_transform, camera_n @ world_transform)
    camera /= np.linalg.norm(camera, ord="fro")
    if camera.flat[np.argmax(np.abs(camera))] < 0:
        camera = -camera
    homogeneous = np.vstack((world, np.ones(world.shape[1])))
    projected_h = camera @ homogeneous
    if np.any(np.abs(projected_h[2]) < 1e-12):
        raise ValueError("A supplied point projects to infinity.")
    projected = projected_h[:2] / projected_h[2]
    errors = np.linalg.norm(projected - image, axis=0)
    return camera, projected, errors, singular_values, rank


def main():
    np.set_printoptions(precision=10, suppress=True, linewidth=90)
    camera, projected, errors, singular_values, rank = estimate_camera()
```

```
    print("Q5: normalized DLT from the six given correspondences")
    print("P (Frobenius norm = 1; largest coefficient positive):")
    print(camera)
    print(f"Frobenius norm: {np.linalg.norm(camera, 'fro'):.12f}")
    print(f"Normalized design matrix rank: {rank} / 12")
    print("Normalized design singular values:")
    print(singular_values)
    print("Reprojected pixels (columns match the assignment):")
    print(projected)
    print(f"Maximum reprojection error (px): {errors.max():.3e}")
    print("P / P[2, 3] (a readable representative):")
    print(camera / camera[2, 3])


if __name__ == "__main__":
    main()
```

Appendix B: complete Python code for Question 6

File: `experients/assignment1/q6_triangulation.py`. This is the same source file that generated the output in Question 6.

```
"""Question 6: homogeneous linear triangulation with the supplied matrices."""

import numpy as np


P1 = np.array([
    [900, 0, 400, 0],
    [0, 900, 250, 0],
    [0, 0, 1, 0],
], dtype=float)
P2 = np.array([
    [900, 0, 400, -6],
    [0, 900, 250, 0],
    [0, 0, 1, 0],
], dtype=float)
PIXEL1 = np.array([400, 250], dtype=float)
PIXEL2 = np.array([385, 250], dtype=float)


def triangulate(p1=P1, p2=P2, pixel1=PIXEL1, pixel2=PIXEL2):
    """Stack x P_3 - P_1 and y P_3 - P_2 for each view."""
    design = np.vstack([
        pixel1[0] * p1[2] - p1[0],
        pixel1[1] * p1[2] - p1[1],
        pixel2[0] * p2[2] - p2[0],
        pixel2[1] * p2[2] - p2[1],
    ])
    _, singular_values, vh = np.linalg.svd(design)
    if np.linalg.matrix_rank(design) != 3:
        raise ValueError("The correspondence does not determine a unique point.")
    point_h = vh[-1]
    if abs(point_h[3]) < 1e-12:
        raise ValueError("The triangulated point is at infinity.")
    point_h = point_h / point_h[3]
    projected_h = np.vstack((p1 @ point_h, p2 @ point_h))
    if np.any(np.abs(projected_h[:, 2]) < 1e-12):
        raise ValueError("Projection has a zero denominator.")
    projected = projected_h[:, :2] / projected_h[:, 2, None]
    errors = np.linalg.norm(projected - np.vstack((pixel1, pixel2)), axis=1)
    return point_h, projected, errors, singular_values


def main():
    np.set_printoptions(precision=10, suppress=True, linewidth=90)
    point_h, projected, errors, singular_values = triangulate()
```

```
    print("Q6: homogeneous linear triangulation")
    print("3D point [X, Y, Z]:")
    print(point_h[:3])
    print("Homogeneous point [X, Y, Z, 1]:")
    print(point_h)
    print("Design singular values:")
    print(singular_values)
    print("Reprojected pixels (rows: view 1, view 2):")
    print(projected)
    print(f"Maximum reprojection error (px): {errors.max():.3e}")
    print(f"Depth in both cameras: {point_h[2]:.10f}")
    print(f"Independent disparity depth 6 / (400 - 385): {6 / 15:.10f}")


if __name__ == "__main__":
    main()
```

Appendix C: verification and reproducibility

The experiments require Python, NumPy and SymPy. From the repository root:

```
python -m pip install -r experients/assignment1/requirements.txt
python experients/assignment1/run.py
powershell -NoProfile -File report/build.ps1
```

The build script reruns the experiments and compiles this PDF with XeLaTeX. The source for the independent checks is `experients/assignment1/verify.py`. It uses exact rational arithmetic for the camera's rank and null vector, exact reprojection for triangulation, and independent projected points to check the epipolar convention. Random checks use seed 1050; the numerical checks supplement the proofs above. Its actual printed results are:

```
PASS: Q1: parallel line has vanishing point (4/5, 2/5)
PASS: Q1: parallel line has vanishing point (4/5, 2/5)
PASS: Q2: independent plane directions give line (6, 3, 2)
PASS: Q3: the plane normal equals four times the given vector
PASS: Q4: 100 rigid motions preserve distance; max error 8.882e-16
PASS: Q5: exact rational DLT has rank 11 and the stated null vector
PASS: Q5: computed unit-norm camera agrees with exact solution and all six pixels
PASS: Q6: triangulation agrees with disparity depth and reprojects in both views
PASS: Q6: exact reprojection gives [400, 250]
PASS: Q6: exact reprojection gives [385, 250]
PASS: Q7: E is the rank-two skew matrix of translation (3, 0, 0)
PASS: Q8: epipolar line is (0, -3, 600), or y = 200
PASS: Q7/Q8: p2.T E p1 = 0 for 20 independently projected 3D points
PASS: Q9: squared error = 117362.5; error = 342.5821069466 px
PASS: Q10: unit-scale orthographic image is (5, 4)
All checks passed. Numerical checks supplement the written proofs.
```

Recorded execution environment

Component	Recorded value
Python	3.10.16
NumPy	2.2.6
SymPy	1.13.3
Manifest SHA-256 (first 16 hex)	`8d96f70449b47602`

The complete SHA-256 hashes of the assignment inputs, code, report sources and printed outputs are in `experients/assignment1/results/manifest.json`. The file `experients/assignment1/inputs.md` transcribes the numerical inputs and records the submission requirements. Original task sources are `assignment1_CS1050.pdf` and the accompanying submission notice. Rounded matrix displays may contain signed zeros; validation uses the unrounded arrays. The final PDF includes both complete question-specific Python files and their printed results, so it can be submitted as one file.